Сравнение и анализ оптимизаций компиляторов

Чочиев Ацамаз Эдгарович

Группа: *25.Б73-мм*
Кафедра: *системного программирования*
Научный руководитель: *Гориховский Вячеслав Игоревич*
Номер семестра практики: *2*

1. Постановка задачи

Работа посвящена исследованию стандартных оптимизаций компиляторов языка C применительно к программному обеспечению встраиваемых и промышленных контроллеров. В таких системах важны как быстродействие, так и размер исполняемого кода, поэтому один и тот же исходный текст может требовать разных режимов компиляции в зависимости от ограничений целевой платформы. **Целью работы является** сравнение того, как GCC, Clang и Tiny C Compiler (TCC) изменяют ассемблерный код, размер программы и время её выполнения при использовании стандартных уровней оптимизации.

Для достижения цели требовалось изучить предоставленные материалы ARTICLE.txt и optbench.c, выбрать основной и два дополнительных компилятора, зафиксировать параметры экспериментальной системы, получить сборки без оптимизации и с оптимизацией по скорости и размеру, сравнить полученный ассемблерный код, а затем повторить эксперимент на собственных тестах, характерных для выбранного сценария.

Результаты работы полезны при выборе компилятора и режима сборки для программ, работающих в условиях ограниченной памяти или жёстких требований к производительности. В роли заказчика выступает учебная практика кафедры системного программирования: итогом должен быть воспроизводимый эксперимент, по которому можно обосновать выбор компилятора и конкретного уровня оптимизации.

2. Описание предлагаемого решения

В качестве основного компилятора выбран GCC 13.3.0. Для сравнения использованы Clang 18.1.3 и Tiny C Compiler 0.9.27. Эксперименты проводились в Linux Mint 22.2 на архитектуре x86-64, ядро 6.14.0-29-generic, процессор Intel Core i5-12450H.

Основной эксперимент построен вокруг программы optbench.c, в которой отдельные участки кода демонстрируют типовые оптимизации компилятора. Для GCC и Clang исследованы уровни `-O0`, `-O1`, `-O2`, `-O3` и `-Os`. Ассемблерный код получался с помощью `-S`, а для унифицированного анализа исполняемых файлов применялся `objdump`. Размеры секций измерялись утилитой `size`; производительность – `hyperfine` с серией повторных запусков. Сборка и сбор результатов автоматизированы shell-скриптами.

Для проверки выводов на коде, связанном с выбранным сценарием, дополнительно разработаны четыре небольших теста: операции с битовыми масками (модель работы с регистрами контроллера), кольцевой буфер для потоковых данных, CRC32 для проверки целостности данных и целочисленный фильтр низких частот для обработки показаний датчиков. Для этих программ также сравнивались размеры, время выполнения и ассемблерный код.

Дополнительно реализованы тесты роста памяти для двух способов хранения данных: локального буфера в стеке и динамического буфера, выделяемого через `malloc` в куче. Размер буфера изменяется от 1 до 256 КиБ; тесты собираются GCC, Clang и TCC с режимами `-O0`, `-O2` и `-Os`. Всего сформировано 126 строк результатов (комбинации компилятора, режима оптимизации, размера и типа памяти). Для GCC и Clang размер стекового кадра дополнительно считывается из отчётов компилятора о расходе стека; для TCC это поле не заполняется, поскольку используемый способ измерения к нему неприменим.

3. Эксперименты

На основной программе optbench.c режим `-Os` дал минимальный размер секции `.text`: 3021 байт для GCC и 3029 байт для Clang. Для GCC переход от `-O0` к `-Os` уменьшил `.text` с 3760 до 3021 байта (примерно на 19.7%), для Clang – с 3492 до 3029 байт (примерно на 13.3%). При `-O3` размер кода для обоих компиляторов немного возрастал относительно `-O2`, что показывает ожидаемый компромисс между скоростью и компактностью.

Итоговые измерения собственных тестов приведены в таблице. Время – среднее по 20 измерениям после трёх прогревочных запусков; после знака $\pm$ указано стандартное отклонение.

Тест	GCC, лучший вариант	Clang, лучший вариант	TCC, `-O2`
Bitmask	17.4 ± 0.3 мс (`-O2`)	5.6 ± 0.6 мс (`-O2`)	20.9 ± 1.3 мс
Ring buffer	4.5 ± 0.8 мс (`-O2`)	5.9 ± 1.3 мс (`-O2`)	18.6 ± 1.6 мс
CRC32	64.3 ± 0.7 мс (`-O2`)	32.3 ± 0.7 мс (`-O2`)	91.5 ± 3.6 мс
Low-pass filter	5.2 ± 0.3 мс (`-O2`)	6.6 ± 0.9 мс (`-Os`)	11.7 ± 0.7 мс

Наиболее заметный эффект получен в CRC32: Clang уменьшил среднее время с 210.0 ± 8.9 мс при `-O0` до 32.3 ± 0.7 мс при `-O2`, то есть примерно в 6.5 раза. При этом размер секции `.text` вырос с 1689 до 1756 байт. Это наглядный пример оптимизации, при которой рост объёма кода оправдан существенным повышением производительности. В тесте кольцевого буфера лучший результат дал GCC `-O2`, а в тесте битовых операций – Clang `-O2`.

Анализ ассемблера показал, что GCC и Clang встраивают небольшие функции, активнее держат промежуточные значения в регистрах и преобразуют циклы. В bitmask Clang выполнил более агрессивное преобразование цикла, что соответствует заметному ускорению. В CRC32 Clang сильнее преобразовал последовательность битовых операций, тогда как GCC сохранил более простой внутренний цикл. TCC генерировал код, близкий по структуре к исходному C: сохранялись отдельные вызовы функций и частые обращения к стеку. Для TCC варианты `-O0` и `-O2` практически не отличались по размеру и времени выполнения.

Из эксперимента следует, что универсального победителя нет. GCC показывает устойчиво хорошие результаты на разных типах нагрузки, Clang в отдельных вычислительных тестах оптимизирует агрессивнее и может быть существенно быстрее, а `-Os` у GCC и Clang обычно обеспечивает наиболее компактный код. Выбор режима должен определяться ограничениями конкретной системы.

Тесты роста памяти дополняют сравнение производительности анализом размера используемых буферов. График стека показывает размер стекового кадра, сообщённый компилятором там, где такой отчёт доступен; это не измерение полного RSS процесса. График кучи показывает запрошенный размер выделения `malloc`, а не фактически занятую физическую память. Исходники и результаты доступны в репозитории: тест стека, тест кучи, скрипт запуска, таблица CSV, график стека и график запрошенного размера кучи.

4. Заключение

В ходе выполнения работы получены следующие результаты:

- проведено сравнение GCC 13.3.0, Clang 18.1.3 и TCC 0.9.27 на общей экспериментальной системе;
- исследовано влияние уровней `-O0`, `-O1`, `-O2`, `-O3` и `-Os` на размер и ассемблерный код основной тестовой программы;
- выполнены повторяемые измерения производительности и размеров кода;
- разработаны четыре дополнительных теста для сценария встраиваемых и промышленных контроллеров;
- добавлены тесты роста памяти в стеке и куче, результаты которых представлены в CSV и графиках;
- проведён сравнительный анализ конкретных ассемблерных преобразований GCC, Clang и TCC;
- подготовлены скрипты автоматической сборки и измерений, а также CI-проверка репозитория.

Результаты показывают, что оптимальный компилятор и уровень оптимизации зависят от типа нагрузки и ограничений целевой системы. В проведённых измерениях `-Os` позволял уменьшить размер секции `.text`, а `-O2` часто обеспечивал хорошее соотношение скорости и размера кода. Clang оказался особенно эффективен в тесте CRC32, тогда как GCC показал конкурентные результаты на остальных задачах. Поэтому выбор следует подтверждать измерениями на целевой платформе, а не только общими рекомендациями.

Тесты памяти дополняют этот вывод, но измеряют разные характеристики: для GCC и Clang график стека отражает размер стекового кадра по отчёту компилятора, а график кучи – запрошенный приложением размер `malloc`. Эти показатели не являются измерением полного потребления физической памяти процесса. Для практического применения важно учитывать также ограничения стека, накладные расходы аллокатора и поведение конкретной операционной системы.

Материалы, иллюстрирующие выполнение работы, размещены в релизе: `https://github.com/atsa111/compiler-optimization-study/releases/tag/v1.0.0`.

Репозиторий: `https://github.com/atsa111/compiler-optimization-study`.

Техническая документация и подробные результаты экспериментов находятся в README, каталогах `results/` и `asm/`, а также в полном отчёте репозитория. Работа завершена в рамках учебной практики; итоговая версия отмечена релизом `v1.0.0`, а основная ветка проходит автоматическую CI-проверку.

5. Список литературы и документации

1. GNU Project. *GCC: Optimize Options*. Официальная документация по уровням оптимизации. `https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html`
2. LLVM Project. *Clang Compiler User's Manual*. Официальное руководство по Clang и параметрам компиляции. `https://clang.llvm.org/docs/UsersManual.html`
3. Bellard, F. *Tiny C Compiler (TCC)*. Официальный сайт и документация компилятора. `https://bellard.org/tcc/`
4. Sharkdp. *hyperfine: A command-line benchmarking tool*. Документация инструмента измерения времени выполнения. `https://github.com/sharkdp/hyperfine`
5. GNU Binutils. *objdump – display information from object files*. Документация по анализу объектных файлов и ассемблерного кода. `https://sourceware.org/binutils/docs/binutils/objdump.html`
6. Linux man-pages project. *malloc(3) – allocate and free dynamic memory*. Описание динамического выделения памяти. `https://man7.org/linux/man-pages/man3/malloc.3.html`
7. Chochiev A. E. *compiler-optimization-study*. Исходный код тестов, скрипты и результаты экспериментов. `https://github.com/atsa111/compiler-optimization-study`